

Operating Systems — 歷屆考古題彙整（2011–2016）

題目文字直接取自原始考卷 PDF，原封不動。相同題目已合併，括號標注出現年份。答案請見 [考古題解答.md](#)

壹、中斷與 I/O 機制

Q1 | Interrupt-based vs Busy-waiting

(2011 Q1a / 2013 Q1a / 2014 Q1a)

Show the advantage of interrupt-based operations over busy-waiting.

Q2 | Hardware Interrupt Handling

(2011 Q1b / 2012 Q1b / 2013 Q1b / 2014 Q1b)

Describe the process of hardware interrupt handling. You need to mention **device controller**, **interrupt-request line**, **state saving**, **interrupt vector**, **interrupt-specific handler**.

Q3 | Modern Interrupt Mechanism

(2016 Q1a)

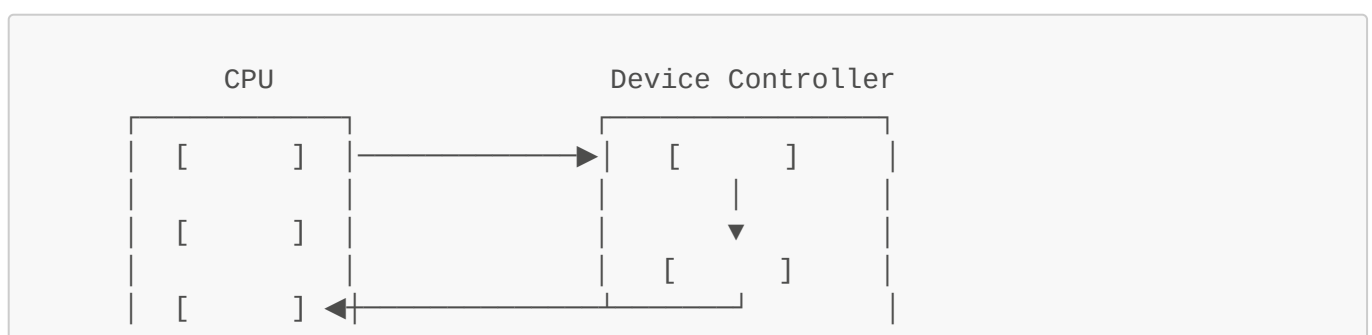
How does the modern interrupt mechanism accelerate an interrupt operation?

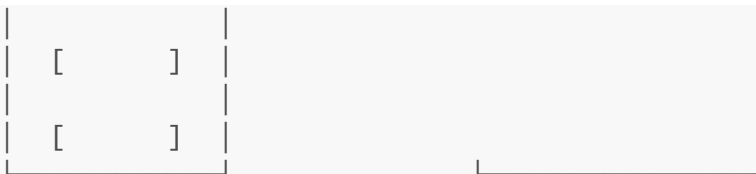
Q4 | Interrupt-driven I/O Cycle Flowchart

(2015 Q1b / 2016 Q1b)

The interrupt-driven I/O cycle consists of (i) Input ready, output complete, or error generates interrupt signal, (ii) Interrupt handler processes data, return from interrupt, (iii) CPU executing checks for interrupts between instructions, (iv) device driver initiates I/O, (v) Controller initiates I/O, (vi) CPU resumes processing of interrupted process (task), and (vii) CPU receiving interrupt, transfer control to interrupt handler.

Organize the steps with the following flowchart.





Q5 | Interrupt Vector 的好處

(2011 Q1c / 2014 Q1c)

Explain the benefit of using interrupt vector.

Q6 | Device Controllers and Device Drivers

(2013 Q2 / 2015 Q1a)

What role do device controllers and device drivers play in a computer system?

Q7 | I/O Devices and CPU Execute Concurrently

(2012 Q1a)

What do we mean I/O devices and the CPU execute concurrently?

貳、系統結構 (System Structures)

Q8 | API Function Invoking a System Call

(2011 Q3 / 2012 Q2a / 2013 Q4a / 2014 Q4a / 2015 Q4a / 2016 Q2a)

Describe how an API function invokes a system call. You **need to mention** *software interrupt*, *dual-mode*, *system call number (index)*, and *table of code pointer*.

2011 原文：Invoking system calls is tedious, so application programmers prefer using API functions to access the services provided by operating systems. Describe how an API function invokes a system call on behavior of a programmer. You need to mention software interrupt, dual-mode, system call number (index), and table of code pointer.

Q9 | Why Prefer API over Direct System Calls

(2011 Q3 / 2012 Q2b / 2013 Q4b / 2014 Q4b / 2015 Q4b / 2016 Q2b)

Explain why programmers prefer programming according to API functions rather than invoking system calls directly.

2012/2013 原文：Explain why programs mostly access system services via API functions rather than making system call directly.

Q10 | Passing MANY Parameters to OS

(2013 Q4c / 2014 Q4c)

- **2013** : Describe an appropriate method used to pass "**MANY**" parameters to the operating system during system calls.
- **2014** : Describe a method to pass system call parameters to the operating system.

Q11 | Layered OS Structure — Pros and Cons

(2011 Q4 / 2016 Q2c)

Describe the pros and cons of layered operating system structure.

Q12 | Microkernel System Structure — Pros and Cons

(2012 Q3)

Describe the pros and cons of the microkernel system structure.

Q13 | Modular Kernel — Advantages over Layered and Microkernel

(2013 Q5)

What are the advantages of the modular kernel approach over the layered and microkernel design techniques?

Q14 | Dual-mode Operation

(2012 Q1d / 2013 Q3)

- **2012** : What is dual-mode? How does it ensure the proper execution of operating systems?
- **2013** : What is the dual-mode operation and how does it ensure the proper execution of the operating system?

Q15 | Higher-level Language for OS Implementation

(2015 Q2)

What are the advantages of using a higher-level language to implement an operating system?

Q16 | CLI — Two Implementation Approaches

(2015 Q3)

There are two different ways that commands can be processed by a command interpreter. One way is to allow the command interpreter to contain the code needed to execute the command. The other way is to implement the commands through system programs. Compare and contrast the two approaches.

Q17 | Multiprogramming vs Timesharing**(2011 Q2 / 2012 Q1c / 2016 Q1c)**

- **2011** : Explain the difference between multiprogramming and timesharing.
 - **2012** : What is time-sharing? Explain the difference between it and multi-programming.
 - **2016** : What is the benefit of multi-programming? Explain the difference between it and time-sharing.
-

Q18 | Asymmetric vs Symmetric Multiprocessing**(2014 Q2)**

What is the difference between asymmetric multiprocessing and symmetric multiprocessing?

Q19 | How OS Supports New Devices**(2014 Q3 / 2016 Q1d)**

How do operating systems support new devices?

參、行程 (Processes)

Q20 | Four Components of a Process in Memory**(2014 Q5)**

A process in memory generally consists of four major components. What are the components and what is the use of each component?

Q21 | Process State — Situation判斷**版本 A (2011 Q5)**

Show the state of a process when the process:

Situation	State
(a) (3 pts) is selected by a long-term scheduler	

Situation	State
(b) (3 pts) is selected by a short-term scheduler	
(c) (3 pts) invokes a blocking system call	
(d) (3 pts) completes a blocking system call	
(e) (3 pts) is interrupted by an interrupt	

版本 B (2012 Q4 / 2013 Q6)

Show the state of a process:

Situation	State
(a) (3 pts) after it is selected by a long-term scheduler	
(b) (3 pts) before it is selected by a short-term scheduler	
(c) (3 pts) after it is selected by a short-term scheduler	
(d) (3 pts) before it invokes a blocking system call	
(e) (3 pts) after it invokes a blocking system call	
(f) (3 pts) after the blocking system call is complete	
(g) (3 pts) after it is interrupted by a hardware interrupt	

版本 C (2015 Q5)

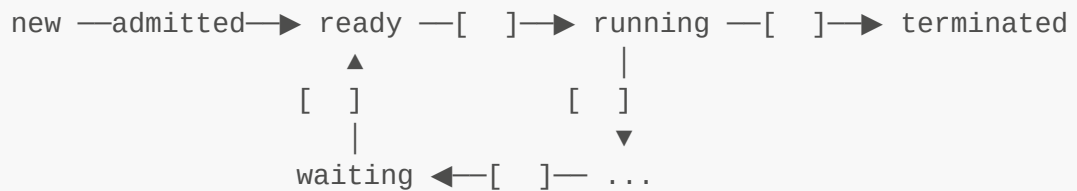
Show the state of a process:

Situation	State
(a) (2 pts) after it is created (forked) by a (parent) process	
(b) (2 pts) before it is selected by a short-term scheduler	
(c) (2 pts) after it is selected by a short-term scheduler	
(d) (2 pts) after it invokes a blocking system call	
(e) (2 pts) after the blocking system call is complete	
(f) (2 pts) after it is interrupted by a hardware interrupt	
(g) (3 pts) after its time slice expired	

Q22 | Process State Diagram — Fill in Transitions

(2016 Q3a)

During its life cycle, a process can (i) be interrupted, (ii) be dispatched by a short-term scheduler, (iii) perform I/O, (iv) have I/O completed, (v) exit. Organize these operations with the following diagram.



Fill in each arrow label with the correct operation from (i)–(v):

```

ready    → running    : (    )
running  → ready      : (    )
running  → waiting     : (    )
waiting  → ready       : (    )
running  → terminated : (    )
  
```

Q23 | fork() Output Analysis

版本 A — 2016 Q3b

(5 points) Show the output produced by the following program, and (5 points) explain what would happen if we remove the statement `wait(NULL)`?

```

int value = 5; // value is a global variable

int main(int argc, char *argv[])
{
    pid_t pid;

    pid = fork();

    if (pid > 0) {
        value += 15;
        wait(NULL);
        printf("value = %d\n", value);
    } else if (pid == 0) {
        value += 5;
        printf("value = %d\n", value);
    }
}
  
```

版本 B — 2012 Q5

(5 points) Show the output of the following program:

```
int main(int argc, char *argv[])
{
    pid_t pid;
    int i = 0;

    pid = fork();
    i = i + 5;
    if (pid == 0) {
        i = i + 5;
        printf("i = %d\n", i);
    } else {
        wait(NULL);
        printf("i = %d\n", i);
    }

    pid = fork();
    i = i + 5;
    if (pid == 0) {
        i = i + 5;
        printf("i = %d\n", i);
    } else {
        wait(NULL);
        printf("i = %d\n", i);
    }
    return 0;
}
```

Q24 | Shared Memory — Calculate $\Sigma n!$

(2011 Q6 / 2014 Q9)

(20 pts / 10 pts) Complete the following program which calculates $\Sigma(n=1 \text{ to } N) n!$. The value N is provided by users in the command line, i.e., $N = \text{atoi}(\text{argv}[1])$.

Hint: The size of an integer is 4 bytes. The program needs to create N child processes — one for each factorial calculation (i.e., $n!$). The child processes save the results in a shared integer memory and the parent process prints the final outcome. To prevent inconsistent results, parent needs to wait for each child process.

```
#include <sys/shm.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    int N = atoi(argv[1]);
```

```

pid_t pid;
int segment_id;
int *shared_memory;

// allocate and attach a shared memory segment
__(A)___
__(B)___

__(C)___ // initialization

for (__(D)___) // create child processes
{
    __(E)___

    if (__(F)___) {
        fprintf(stderr, "Fork Failed");
        exit(-1);
    } else if (__(G)___) { // child part
        // do factorial calculation and save it
        __(H)___
        // then detach shared memory and exit
        __(I)___
    } else { // parent part
        __(J)___
    }
}

printf("the result is %d\n", *shared_memory);

// detach and remove the shared memory segment
__(K)___
__(L)___

return 0;
}

```

Q25 | Shared Memory — Calculate $2x+3y-5$

(2012 Q6 / 2015 Q9)

(15 points) Please complete the following program that calculates equation $2x+3y-5$. The values of variables x and y are provided by users in the command line, i.e., $x = \text{atoi}(\text{argv}[1])$ and $y = \text{atoi}(\text{argv}[2])$.

Hint: the main program first allocates a shared memory and then creates two child processes — one for the x term calculation and one for the y term calculation. Save the results calculated in the child processes in the shared memory and print the final outcome in the parent process.

```

#include <sys/shm.h>
#include <sys/stat.h>
#include <stdio.h>

```



```
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    int x = atoi(argv[1]); // the value of variable x
    int y = atoi(argv[2]); // the value of variable y
    int i;
    pid_t pid;
    const int sh_size = 4; // shared memory size = 4 bytes

    // allocate a shared memory segment
    int segment_id = __ (A) __ (IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR);
    // attach the shared memory segment
    int *shared_memory = (int *) __ (B) __ (segment_id, NULL, 0);

    *shared_memory = __ (C) __; // initialization

    for (i = 2; i > 0; i--) // create two child processes
    {
        pid = __ (D) __ ();

        if (pid < 0) {
            fprintf(stderr, "Fork Failed");
            exit(-1);
        } else if (pid == __ (E) __) { // child part
            if __ (F) __ // do x term calculation
            if __ (G) __ // do y term calculation

            // detach the shared memory segment and exit
            shmdt(shared_memory);
            exit(0);
        } else { // parent part
            __ (H) __;
        }
    }

    *shared_memory -= 5;
    printf("the result is %d\n", *shared_memory);
    // detach the shared memory segment
    shmdt(shared_memory);
    // remove the shared memory segment
    __ (I) __ (segment_id, IPC_RMID, NULL);

    return 0;
}
```

Q26 | Shell Interface Program

(2013 Q8 / 2016 Q5)

(15 points) Complete the following C++ program which serves as a simple shell interface. The shell accepts user commands and then executes each command in a separate process. The shell allows the child process to run in the background (or concurrently) if the command ends with `&`. For simplicity, we assume that users never enter invalid commands and no arguments follow a command.

```
#include <iostream>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
using namespace std;

int main(void)
{
    char inputBuffer[80]; // buffer to hold command entered
    bool background;      // true if a command is followed by '&'

    while (__(A)__)
    {
        cout << endl << " COMMAND->";
        pid_t pid;

        // step 1
        // accept user command and check if the command is followed by '&'
        __(B)__

        // step 2
        // fork a child process, if background == false, the parent will
        wait
        // the child process invoke exec(inputBuffer) to execute user
        command
        __(C)__

    } // end of while
} // end of main
```

肆、執行緒 (Multithreaded Programming)

Q27 | Why Process Creation is Costly vs Thread Creation

(2015 Q6a / 2016 Q4a)

Explain why allocating memory for process creation is **costly** as compared with thread creation.

Q28 | Many-to-One Thread Model — Pros and Cons

(2012 Q7a / 2013 Q7a)

Describe the pros and cons of the many-to-one thread model.

Q29 | One-to-One Thread Model — Pros and Cons

(2013 Q7b / 2016 Q4b)

Describe the pros and cons of the one-to-one thread model.

Q30 | Compare Many-to-One and One-to-One

(2015 Q6b)

Compare and contrast the many-to-one and one-to-one thread models.

Q31 | Scheduler Activation — Problem it Resolves

(2011 Q7a / 2014 Q6a)

- **2011** : What is the problem of many-to-many thread model that scheduler activation tries to resolve?
 - **2014** : Explain the problem of many-to-many thread model without using scheduler activations.
-

Q32 | Scheduler Activation — How it Works

(2011 Q7b / 2012 Q7b / 2014 Q6b)

- **2011 / 2012** : Describe the operation of scheduler activation.
 - **2014** : Show how scheduler activations work when a user thread is making a blocking system call. You **HAVE TO** mention *upcall handler*, *scheduler*, and *thread library*.
-

伍、CPU 排程 (CPU Scheduling)

Q33 | Preemptive Scheduling — Four Situations

(2014 Q7)

Show the four situations under which a preemptive scheduling algorithm takes place. And what is the difference between preemptive scheduling and non-preemptive scheduling?

Q34 | Priority Scheduling — Starvation Solution

(2015 Q7)

Design a solution to avoid the starvation problem of priority scheduling algorithms.

Q35 | Multilevel Feedback Queue — Gantt Chart

(2014 Q8 / 2015 Q8)

Scheduling — Consider the following two processes **P₁** and **P₂**, each of which consists of two CPU bursts and one I/O burst. Assume that P₁ and P₂ arrive at time **0** and **2** (millisecond) respectively:

Process	CPU burst	I/O burst	CPU burst
P ₁	8 ms	4 ms	8 ms
P ₂	2 ms	2 ms	16 ms

(a) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm**. Note that **a high-level process will preempt a low-level process** and **a process will be scheduled to a lower level if it is preempted**.

Round-Robin scheduling
Time quantum = 4 ms.

← highest level

Round-Robin scheduling
Time quantum = 6 ms.

← middle level

FCFS scheduling

← lowest level

Time (ms): 0246810121416182022242628303234

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

Process:

(b) What is the average waiting time for the scheduling algorithm?

Process	Arrival	Finish	Turnaround Time	Waiting Time
P ₁	0			
P ₂	2			

Average Waiting Time = _____ ms

End of 考古題彙整。